

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

Дисциплина: Фронт-энд разработка

Отчет

**Лабораторная работа 3: Разработка одностраничного
веб-приложения (SPA) с использованием фреймворка
Vue.JS**

Выполнили:

**Динкель Алиса
Карнаухова Елизавета
К3344**

**Проверил:
Добряков Д. И.**

Санкт-Петербург

2024 г.

Задача

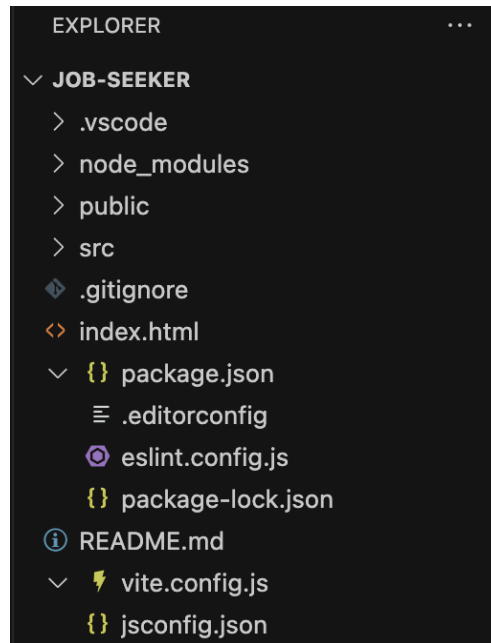
Мигрировать ранее написанный сайт на фреймворк Vue.JS.

Минимальные требования:

- Должен быть подключён роутер
- Должна быть реализована работа с внешним API
- Разумное деление на компоненты
- Использование composable

Ход работы

Структура файла:



Роутер

Подключение роутера в main.js

Для реализации роутинга в проекте был использован официально поддерживаемый пакет vue-router.

В файле main.js создается приложение Vue, и подключаются основные зависимости, включая роутер. Для этого используется метод `app.use(router)`, который активирует роутер на уровне всего приложения:

```
import { createApp } from 'vue'

import App from '@/App.vue'
import router from '@/router'
import store from '@/stores'

import 'bootstrap/dist/css/bootstrap.min.css'
import 'bootstrap'
import '@/assets/main.css'

const app = createApp(App)

app.use(store)
app.use(router)
```

```
app.mount('#app')
```

Конфигурация роутера в src/router/index.js

В файле index.js настроены маршруты приложения. Каждый маршрут связан с конкретным view, который будет отображаться при переходе на указанный путь. Весь роутинг настроен с использованием функции createRouter, которая создаёт экземпляр роутера с историей веб-приложения.

```
import { createRouter, createWebHistory } from 'vue-router'

const router = createRouter({
  history: createWebHistory(import.meta.env.BASE_URL),
  routes: [
    {
      path: '/',
      name: 'main',
      component: () => import('../views/MainPage.vue')
    },
    {
      path: '/login',
      name: 'login',
      component: () => import('../views/LoginPage.vue'),
    },
    {
      path: '/signup',
      name: 'signup',
      component: () => import('../views/SignupPage.vue'),
    },
    {
      path: '/useraccount',
      name: 'useraccount',
      component: () => import('../views/UserAccount.vue'),
    },
    {
      path: '/employeraccount',
      name: 'employeraccount',
      component: () => import('../views/EmployerAccount.vue'),
    },
    {
      path: '/vacancydetails',
      name: 'vacancydetails',
      component: () => import('../views/VacancyDetails.vue'),
    },
  ],
})
```

```
  })  
  
  export default router
```

API

Для работы с внешним API в проекте используется библиотека `axios`, которая позволяет отправлять HTTP-запросы и получать ответы от сервера.

Создание экземпляра `Axios` в `src/api/auth.js`

В файле `auth.js` создается экземпляр `axios` с базовым URL, который указывает на адрес API сервера. Далее описаны методы для регистрации и логина пользователей. Каждый метод отправляет POST или GET запросы на сервер, обрабатывает ответы и ошибки.

```
import axios from 'axios';  
  
const API_URL = 'http://localhost:3000';  
  
class AuthApi {  
  constructor(instance) {  
    this.instance = instance; // Сохраняем экземпляр axios или другой HTTP-клиент  
  }  
  
  async registerUser(registerData) {  
    try {  
      console.log('Регистрация данных:', registerData); // Отладочное сообщение  
      const response = await this.instance.post(`${API_URL}/users`, registerData);  
      console.log('Ответ сервера:', response.data); // Отладочное сообщение  
      return response.data; // Убедитесь, что возвращаете данные  
    } catch (error) {  
      console.error('Ошибка регистрации:', error);  
      throw error; // Пробрасываем ошибку дальше  
    }  
  }  
  
  async loginUser(loginData) {  
    try {  
      const response = await this.instance.get(`${API_URL}/users`, { params:  
loginData });  
      return response.data;  
    }  
  }  
}
```

```

    } catch (error) {
      console.error('Ошибка входа:', error);
      throw error;
    }
  }
}

// Создание экземпляра AuthApi
const instance = axios.create({
  baseURL: API_URL,
});

const authApi = new AuthApi(instance); // Создаем экземпляр AuthApi

export default authApi; // Экспортируем экземпляр

```

Использование Pinia для хранения состояния в authStore.js

Для управления состоянием авторизации используется Pinia. В authStore.js описаны действия для регистрации, логина и выхода пользователя. Все действия по отправке данных и получению ответов инкапсулированы в методы хранилища.

```

import { defineStore } from 'pinia';
import authApi from '@/api/auth'; // Убедитесь, что путь правильный

export const useAuthStore = defineStore('auth', {
  state: () => ({
    user: null,
    loading: false,
    error: null,
  }),

  actions: {
    async login(credentials) {
      this.loading = true;
      this.error = null;
      try {
        const response = await authApi.loginUser(credentials); // Используйте
экземпляр
        this.user = response; // Сохраняем информацию о пользователе
      } catch (error) {
        this.error = error.message; // Записываем ошибку
      } finally {

```

```

        this.loading = false;
    }
},

async register(data) {
    this.loading = true;
    this.error = null;
    try {
        const response = await authApi.registerUser(data); // Вызов метода
регистрации
        console.log('Ответ от сервера при регистрации:', response); // Отладочное
сообщение
        this.user = response; // Сохраняем информацию о новом пользователе
        console.log('Пользователь зарегистрирован:', this.user); // Отладочное
сообщение
    } catch (error) {
        this.error = error.message; // Записываем ошибку
        console.error('Ошибка регистрации:', error); // Выводим ошибку в консоль
    } finally {
        this.loading = false;
    }
},

logout() {
    this.user = null; // Сбрасываем информацию о пользователе
},
},
));

```

Компоненты

Components

В проекте сделано деление на компоненты. Все функциональные единицы, такие как модальные окна, таблицы, карточки и блоки, выделены в отдельные компоненты. Эти компоненты инкапсулируют свою логику и визуальные элементы. Повторяющиеся элементы, такие как шапка, подвал и карточки, также были выведены в отдельные компоненты.

Примеры используемых components:

- HeaderComponent.vue — отвечает за отображение шапки сайта с логотипом и кнопками входа/регистрации или перехода в аккаунт в зависимости от состояния пользователя.

- FooterComponent.vue — отображает информацию о компании и контактные данные.
- LoginCard.vue — форма для входа пользователя с валидацией и обработкой ошибок.

Layouts

Для обеспечения гибкости в размещении контента использованы различные layout-компоненты, представляющие из себя повторяющиеся контейнеры/обертки компонент.

Примеры используемых layouts:

- BaseLayout.vue — используется как обертка для всего содержимого на страницах, обеспечивая основную структуру (body страницы).
- MainLayout.vue — один из повторяющихся контейнеров main.

Views

Каждая страница организована через комбинирование layouts и компонентов. Таким образом, компоненты повторного использования, такие как шапка и подвал, можно легко вставить в различные страницы, а layout-компоненты позволяют организовывать страницу в зависимости от ее структуры.

Пример view:

```
<template>
  <base-layout>
    <HeaderComponent />
    <main-layout>
      <LoginCard />
    </main-layout>
    <FooterComponent />
  </base-layout>
</template>

<script>
import HeaderComponent from "../components/HeaderComponent.vue";
import FooterComponent from "@components/FooterComponent.vue";
import LoginCard from "@components/LoginCard.vue"
import BaseLayout from "@layout/BaseLayout.vue";
import MainLayout from "@layout/MainLayout.vue";

export default {
  name: "MainPage",
  components: {
    HeaderComponent,
    LoginCard,
```



```

    FooterComponent,
    BaseLayout,
    MainLayout,
  },
};
</script>

```

Icons

Также в папке components была использована директория icons, в которую, в свою очередь, были помещены иконки, использующиеся в приложении.

Пример реализации IconAccount:

```

<template>
  <svg xmlns="http://www.w3.org/2000/svg" class="bi bi-person" viewBox="0 0 16 16">
    <path d="M8 8a3 3 0 1 0 0-6 3 3 0 0 0 6 2-3a2 2 0 1 1-4 0 2 2 0 0 1 4 0m4 8c0 1-1 1-1 1H3s-1 0-1-1 1-4 6-4 6 3 6 4m-1-.004c-.001-.246-.154-.986-.832-1.664C11.516 10.68 10.289 10 8 10s-3.516.68-4.168 1.332c-.678.678-.83 1.418-.832 1.664z"/>
  </svg>
</template>

```

Пример использования:

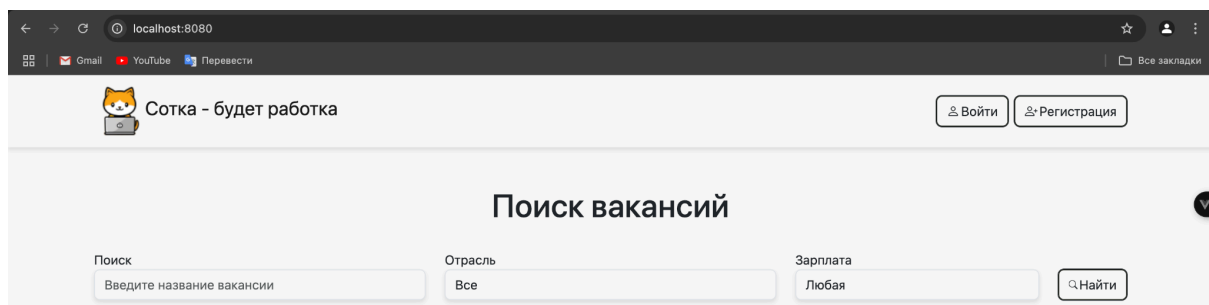
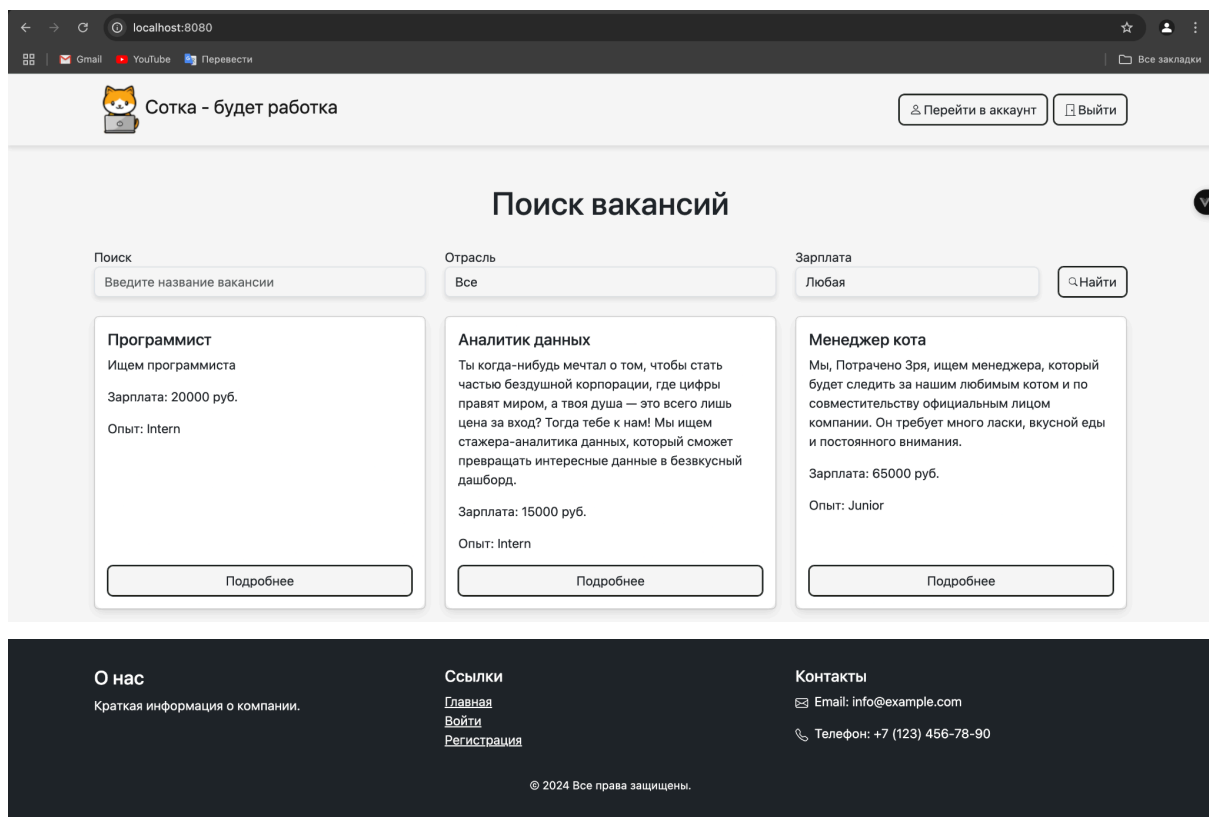
```

<template>
  <div class="row justify-content-center">
    <div class="col-12 col-md-6 col-lg-4 login-container p-4">
      <h3 class="mb-3 text-center">Вход</h3>
      <form @submit.prevent="handleLogin">
        <div class="mb-3">
          <label for="exampleInputEmail1" class="form-label">Логин</label>
          <input type="email"
            class="form-control"
            id="exampleInputEmail1"
            v-model="username"
            required />
        </div>
        <div class="mb-3">
          <label for="exampleInputPassword1" class="form-label">Пароль</label>
          <input type="password"
            class="form-control"
            id="exampleInputPassword1"
            v-model="password"
            required />
        </div>
        <div class="mb-3 form-check">
          <input type="checkbox" class="form-check-input" id="exampleCheck1" />

```

```
        <label class="form-check-label" for="exampleCheck1">Запомнить</label>
    </div>
    <button type="submit" class="btn btn-custom me-2 mb-2">Войти</button>
    <div class="mt-3">
        <router-link to="/signup">Еще нет аккаунта?</router-link>
    </div>
    <div v-if="loginError" class="text-danger mt-2">Неверный логин или
пароль</div>
</form>
</div>
</div>
</template>
```

Внешний вид:



Вывод

В ходе работы разработанный ранее сайт по поиску и размещению вакансий был мигрирован на фреймворк Vue.JS.